

初级数据结构

XJTU 算法竞赛小学期课程 —— Day 6

// Austin4.5

// 2026.7.13



数据结构定义

- 数据结构是一种数据组织、管理和存储的格式。它是相互之间存在一种或多种特定关系的数据元素的集合。

算法竞赛中，能高效组织、存储、查询和修改数据的系统或方法都可称为数据结构。

注：本PPT中的所有代码实现均按照个人习惯编写，大家可参考书上或网上内容来写出其他实现。



目 录

01

简单数据结构

02

集合相关数据结构

03

区间相关数据结构

01



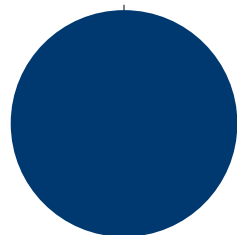
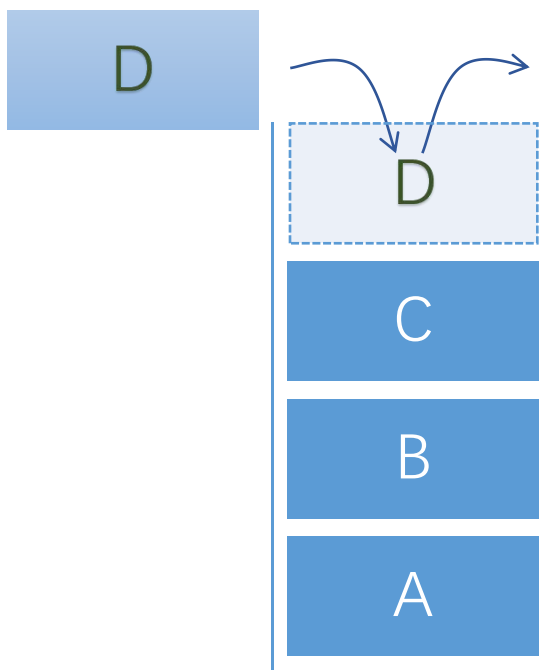
西安交通大学
XI'AN JIAOTONG UNIVERSITY

简单数据结构

- 栈 (stack)
- 队列 (queue)
- 链表 (linklist)

1.1 栈 (stack)

栈是只能在一端进行插入和删除操作的数据结构，遵循后进先出原则。



栈支持的操作

- 向栈顶插入一个元素 $O(1)$
- 从栈顶删除一个元素 $O(1)$
- 查询栈顶元素 $O(1)$



1.1 栈 (stack)

用数组实现

```
const int MAX = 10004;
int st[MAX];
int top = 0;

void push(int val) // 插入一个元素
{
    st[top++] = val;
}

void pop() // 删除一个元素
{
    --top;
}

int get_top() // 查询栈顶元素
{
    return st[top - 1];
}
```

用STL实现

```
#include <stack>

stack<int> st;
st.push(3); // 插入一个元素
st.push(4);
st.pop(); // 删除一个元素
st.top(); // 查询栈顶元素
```

1.1 栈 (stack)

例：平衡括号序列判定

括号序列的定义：仅含 '('、')'、 '[' 和 ']' 的字符串；

平衡括号序列的递归定义：

- (1) 空串是平衡括号序列；
- (2) 如果 A 是平衡括号序列，那么 (A) 和 $[A]$ 是平衡括号序列；
- (3) 如果 A 和 B 是平衡括号序列，那么 AB 是平衡括号序列。

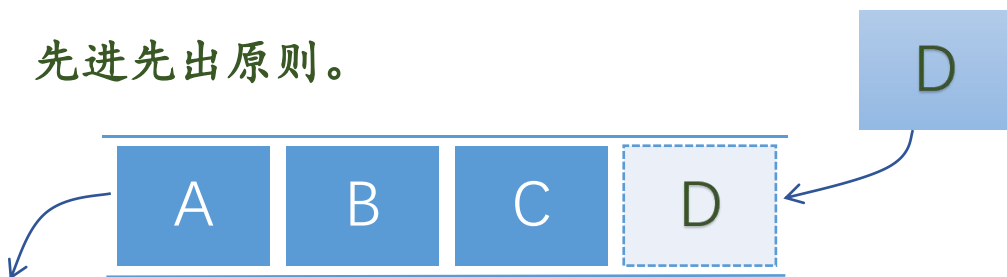
例如：()、[()]、[]([]) 是平衡括号序列，(、[(])、[(] 不是平衡括号序列。

给定一个长度为 $n(n \leq 10^6)$ 的括号序列，判断它是不是平衡括号序列。

做法：从左到右扫描字符串，左括号入栈，右括号和栈顶匹配并弹出，当匹配失败，或者结束时栈仍未空则不是平衡括号序列。

1.2 队列 (queue)

队列是只能在一端进行插入，另一端进行删除操作的数据结构，遵循先进先出原则。



队列支持的操作

- 向队尾插入一个元素 $O(1)$
- 从队首删除一个元素 $O(1)$
- 查询队首/队尾元素 $O(1)$

1.2 队列 (queue)

用数组实现 (循环队列)

```
const int MAX = 10004;
int qu[MAX];
int front = 0, back = 0;

void push(int val) // 插入一个元素
{
    qu[back++] = val;
}

void pop() // 删除一个元素
{
    ++front;
}

int get_front() // 查询队首元素
{
    return qu[front];
}
```

用STL实现

```
#include <queue>

queue<int> qu;
qu.push(3); // 插入一个元素
qu.push(4);
qu.pop(); // 删除一个元素
qu.front(); qu.back(); // 查询队首/队尾元素
```

1.2 队列 (queue)

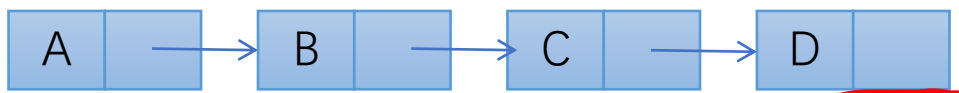
双端队列 (deque)

双端队列支持从其两端插入、删除和修改元素。用STL实现：

```
#include <deque>
deque<int> qu;
qu.push_back(3); // 从队尾插入一个元素
qu.push_back(4);
qu.push_front(5); // 从队首插入一个元素
qu.pop_back(); // 从队尾删除一个元素
qu.pop_front(); // 从队首删除一个元素
qu.front(); qu.back(); // 查询队首/队尾元素
```

1.3 链表 (linklist)

链表是一种物理存储单元上非连续、非顺序的存储结构，由一系列结点组成，每个结点包括两个部分：一个是存储数据元素的数据域，另一个是存储下一个结点地址的指针域。



链表支持的操作

- 寻找一个节点的位置 $O(n)$
- 插入/删除一个节点 $O(1)$

二分插入排序的时间复杂度?

1.3 链表 (linklist)

结构体的定义

```
struct LinkList
{
    int val;
    LinkList* nxt = nullptr;
};
```

一般不用STL list

用数组实现（若没有插入操作）

```
const int MAX = 10004;
int val[MAX];
int nxt[MAX];
int prv[MAX];
```

```
for (int i = 0; i < MAX; ++i)
{
    nxt[i] = i + 1;
    prv[i] = i - 1;
}
prv[0] = -1;
nxt[MAX - 1] = -1;

void remove(int x) // 删除一个节点
{
    nxt[prv[x]] = nxt[x];
    prv[nxt[x]] = prv[x];
}

void do_iteration() // 遍历链表
{
    for (int i = 0; i != -1; i = nxt[i])
    {
    }
}
```

02



西安交通大学
XI'AN JIAOTONG UNIVERSITY

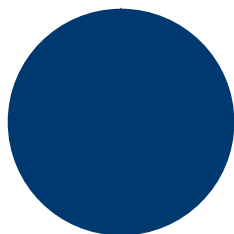
集合相关数据结构

- 有根树 (rooted tree)
- 堆 (heap/priority queue)
- 二叉搜索树 (binary search tree)
- 并查集 (union find)

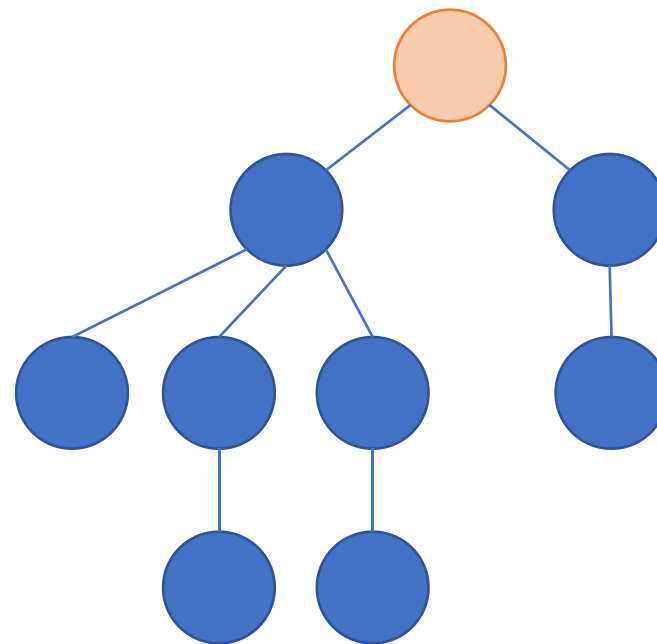
2.0 有根树 (rooted tree)

正式定义：有根树是 $n \geq 0$ 个结点的有限集。当 $n = 0$ 时，称为空树。在任意一棵非空树中应满足：

- (1) 有且仅有一个特定的称为“根”的结点；
- (2) 当 $n > 1$ 时，其余结点可分为若干个互不相交的有限集，其中每一个集合本身又是一颗树，并且称为根的子树。



简单定义：非空树有一个根节点和若干个子树，根节点和所有子树的根节点相连。

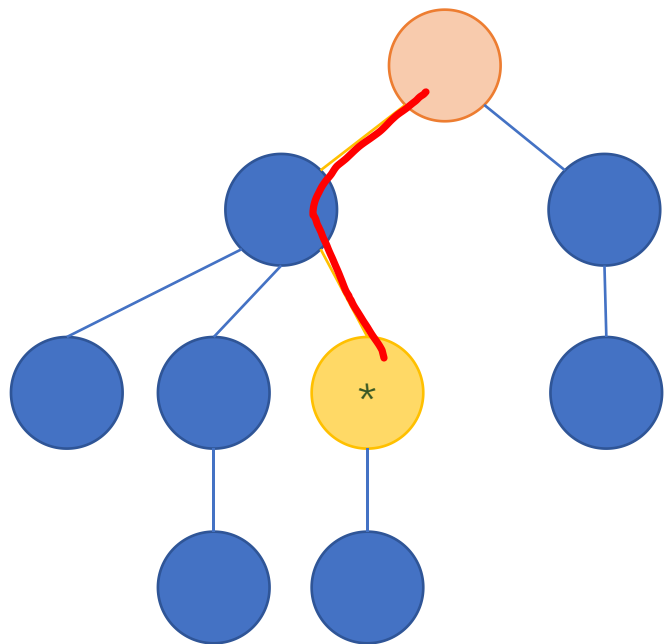


2.0 有根树 (rooted tree)

树的根节点没有前驱，其余节点有唯一的前驱（父节点）。

树的每个节点可以有零个或多个后继（子节点）。

根节点以外的每个节点到根节点都有唯一的路径。



2.0 有根树 (rooted tree)

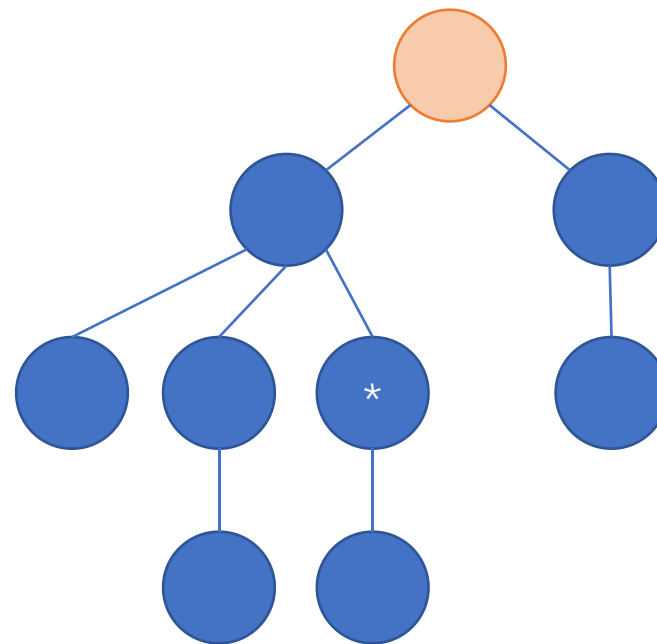
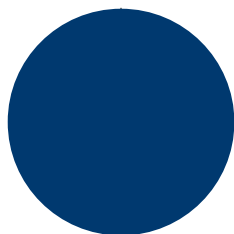
叶子节点：没有后继的节点称为叶子节点。

祖先：一个节点到根节点的路径上除自身以外的所有节点。特别地，根节点没有祖先。

节点的深度：节点到根节点的路径中边的个数。

兄弟：一个节点的子节点互为兄弟。

子树：删去节点和父节点相连的边后，该节点所在的子图。



2.0 有根树 (rooted tree)

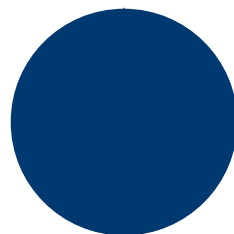
二叉树：每个节点最多有2个子节点，且有左右之分。

二叉树的遍历：

前序遍历：先遍历根节点，再遍历左子树，最后遍历右子树。

中序遍历：先遍历左子树，再遍历根节点，最后遍历右子树。

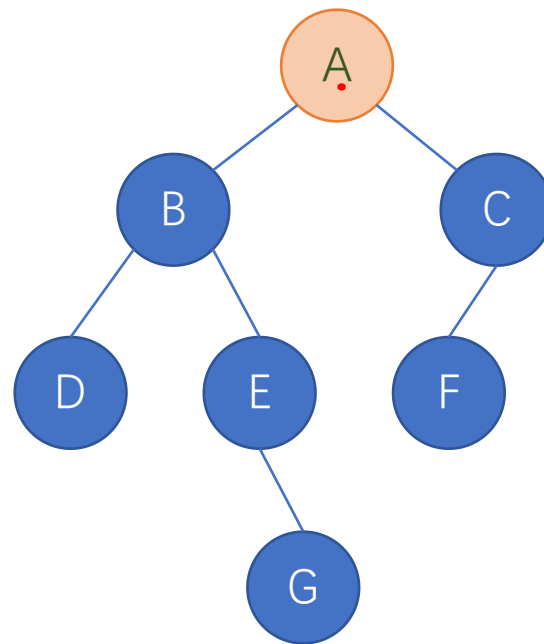
后序遍历：先遍历左子树，再遍历右子树，最后遍历根节点。



前序遍历结果：ABDEGCF

中序遍历结果：DBEGAFC

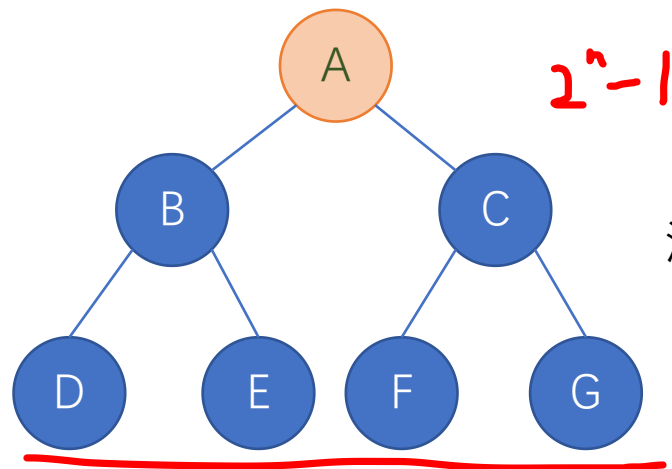
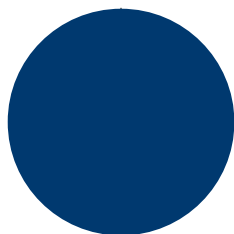
后序遍历结果：DGEBFCA



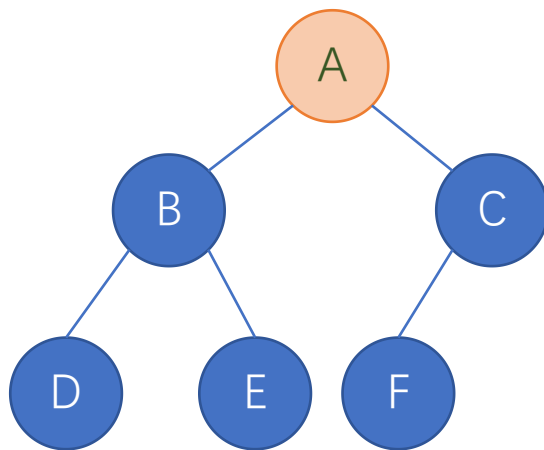
2.0 有根树 (rooted tree)

满二叉树是：所有层节点均填满、叶子全在同一层的二叉树。

完全二叉树：除最后一层外全满、最后一层节点靠左连续排列的二叉树。满二叉树是完全二叉树的特例。



满二叉树



完全二叉树

2.0 有根树 (rooted tree)

二叉树的结构体-指针存储

```
struct TreeNode
{
    int val;
    TreeNode* left = nullptr, * right = nullptr;
};
```

完全二叉树的数组存储

若用零索引来存储完全二叉树，那么第 x 个节点的左右子节点索引分别是 $2x + 1$ 和 $2x + 2$ 。

父节点的索引为 $\lfloor \frac{x-1}{2} \rfloor$ 。

```
const int MAX = 10004;

int tree[MAX];

tree[x]; // 当前节点
tree[2 * x + 1]; // 左子节点
tree[2 * x + 2]; // 右子节点
tree[(x - 1) / 2]; // 父节点
```

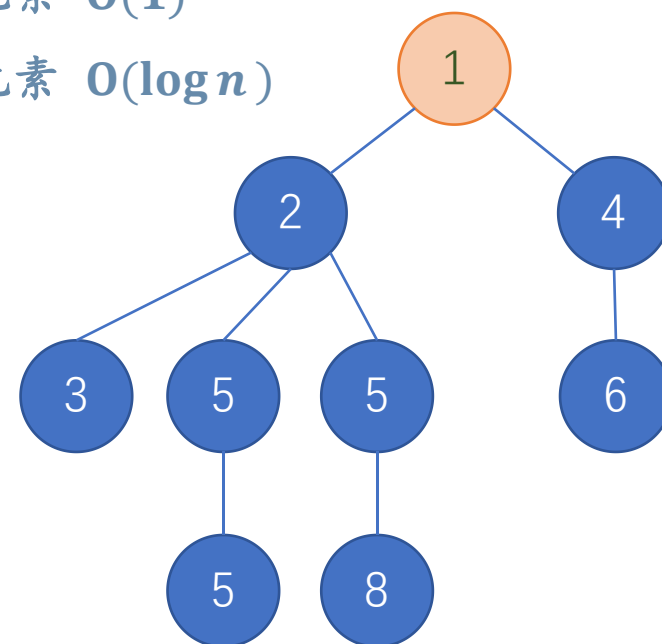
2.1 堆/优先队列 (heap / priority queue)

堆（优先队列）是一棵树，它的每一个节点都有一个值，每个节点的值都大于等于（或小于等于）父节点的值。其中大于等于父节点的称为小根堆，小于等于父节点的称为大根堆。

若树为完全二叉树，则这个堆称为二叉堆。

二叉堆支持的操作

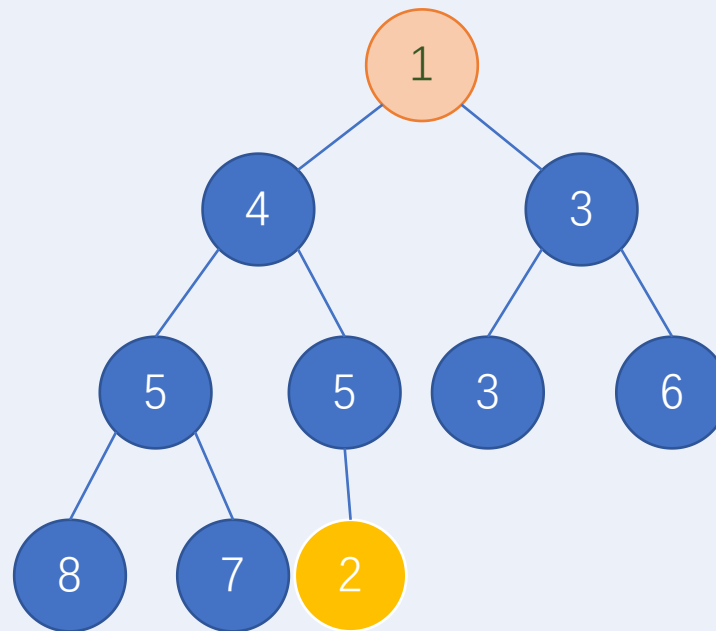
- 向堆中插入一个元素 $O(\log n)$
- 查询最大元素 $O(1)$
- 删除最大元素 $O(\log n)$



2.1 堆/优先队列 (heap / priority queue)

插入元素过程

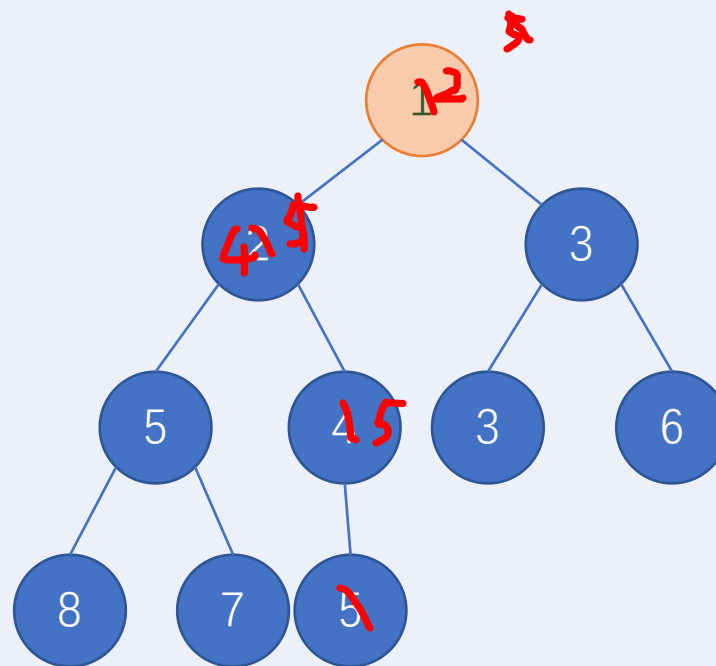
先将元素放入树的末尾部分，使其仍为一个完全二叉树。再依次和其父节点进行比较并考虑交换。



2.1 堆/优先队列 (heap / priority queue)

删除元素过程

删除根节点，然后用最后一个节点替换根节点。
比较其与两个子节点的大小关系，并进行调整。



2.1 堆/优先队列 (heap / priority queue)

优先队列的STL实现

```
#include <queue>

priority_queue<int> pq1; // 定义大根堆
priority_queue<int, vector<int>, greater<int> > pq2; // 定义小根堆
```

堆插入、删除、查询堆顶的操作和栈类似。

2.1 堆/优先队列 (heap / priority queue)

例：中位数维护（洛谷P1168）

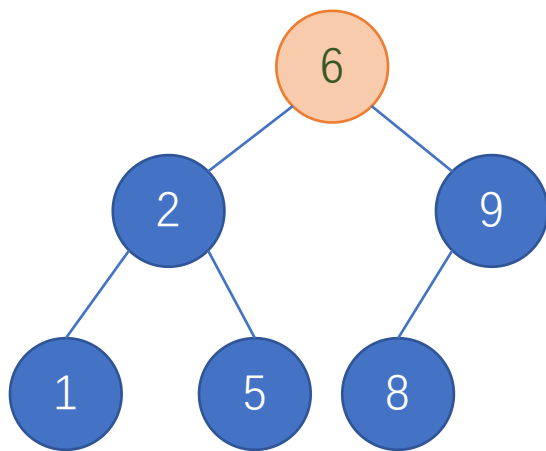
给定长度为 n 的序列，求前奇数项的中位数， $n \leq 10^5$ 。

使用一个大根堆和一个小根堆维护这个序列，其中大根堆放较小的一半数，小根堆放较大的一半数。加入一个数时，比较它和大根堆最大数的大小。需要输出中位数时，将两个堆的大小调整至平衡，最后输出答案即可。

2.2 二叉搜索树 (binary search tree)

二叉搜索树的性质：

- (1) 是二叉树；
- (2) 每个节点若有左子树，则左子树所有节点的值都比这个节点小；若有右子树，则右子树所有节点的值都比这个节点大；
- (3) 二叉搜索树的中序遍历严格递增。



2.2 二叉搜索树 (binary search tree)

插入元素：从根开始，比当前节点元素小就往左走，比它元素大就往右走，走到空位时添加这个节点。

查找元素：方法类似，直到找到这个元素或空节点为止。

删除元素：找到该元素，并找到它的前驱，用前驱替代这个节点，然后递归补足空位。

查找前驱：若有左子节点，则先往左走一步再一直往右走，否则如果当前节点是父节点的右子树，则前驱是父节点；如果是父节点的左子树，则一直往上找祖先，直到某个节点是父节点的右子树为止。



2.2 二叉搜索树 (binary search tree)

例：这是二叉搜索树吗？（团体程序设计天梯赛往年题 L2-004弱化版）

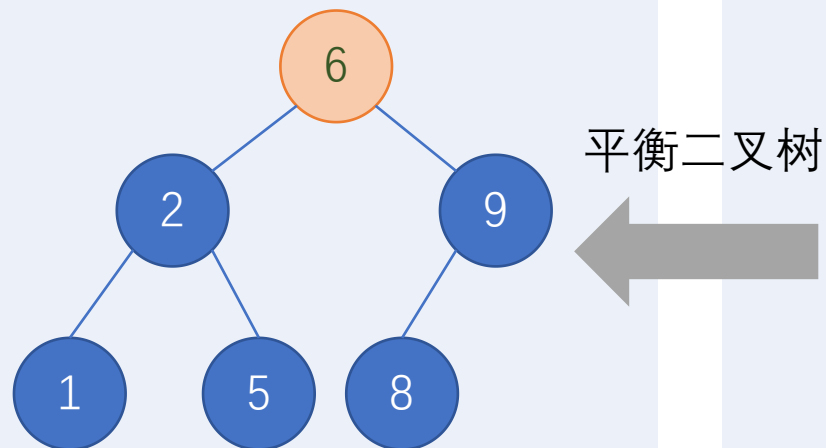
给定一个整数键值序列（ $N \leq 1000$ ），判断这是否是对一棵二叉搜索树进行前序遍历的结果。如果是，则输出其后序遍历。

```
int n;
int arr[MAX];
vector<int> ans;
bool dfs1(int l, int r)
{
    if (l == r) return 0;
    int m = r;
    for (int i = l + 1; i < r; ++i)
    {
        if (arr[i] > arr[l])
        {
            m = i;
            break;
        }
    }
}
```

```
for (int i = m; i < r; ++i)
{
    if (arr[i] < arr[l])
    {
        return 1;
    }
}
if (dfs1(l + 1, m) || dfs1(m, r))
{
    return 1;
}
ans.push_back(arr[l]);
return 0;
}
```

2.2 二叉搜索树 (binary search tree)

以上操作时间复杂度平均 $O(\log n)$,
但最坏达到 $O(n)$



平衡二叉树通过一系列旋转操作，在中序遍历不变时让树的高度维持在 $O(\log n)$ 级别。常见平衡二叉树算法包含红黑树、Treap等。

算法竞赛中很少需要手写平衡树，可以使用STL。

2.2 二叉搜索树 (binary search tree)

set (集合)

```
#include <set>

set<int> s;
s.insert(3); // 插入元素
s.erase(3); // 删除元素
s.find(4); // 查找元素
*(--s.lower_bound(x)); // 查找前驱
*(s.upper_bound(x)); // 查找后继
s.size(); // 求元素个数
s.clear(); // 清空
for (int i : s) {} // 遍历
```

保证升序遍历

multiset (可重集合)

```
#include <set>

set<int> s;
s.erase(3); //
s.erase(s.find(3));
```

其余操作和set类似。

map: 键值无重复映射(字典)

```
#include <map>

map<int, int> mp;
map<int, int>::iterator it = mp.find({ 2, 3 }); // 查找键值对
mp[key]; // 获取键对应的值
mp[2] = 3; // 插入键值
mp.erase(2); // 删除键值
for (pair<int, int> i:mp) { } // 遍历
```

2.2 二叉搜索树 (binary search tree)

例：离散化

数组长度 $n \leq 10^5$ ，值域 $0 \leq a_i \leq 10^9$ ，.....

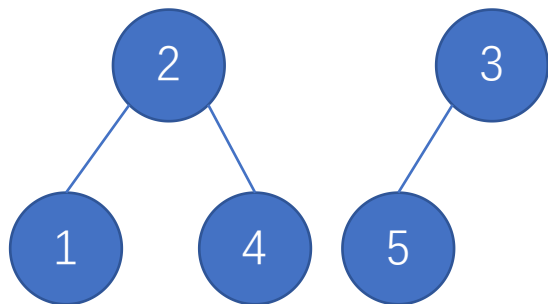
有时我们会在值域上开一个数组。但值域太大，需要使用非常大的空间。

注意到值域内有用的点最多只有 10^5 个，考虑从小到大对这些有用点进行“编号”，用它来代替原数值进行操作。

可使用 set，将所有出现的有用点都放入set中，然后遍历一遍进行编号。

2.3 并查集 (union find)

并查集是一种用于管理元素所属集合的数据结构，实现为一个森林，其中每棵树表示一个集合，树中的节点表示对应集合中的元素。



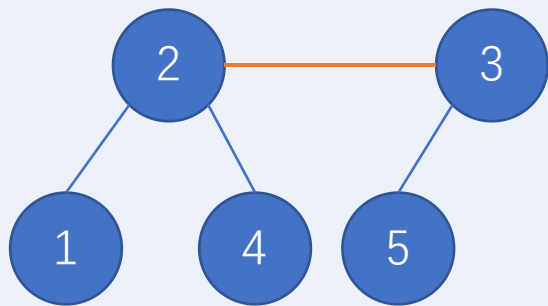
并查集支持的操作

- 初始化 $O(n)$
- 合并两个集合（复杂度后面解释）
- 查询某个元素所属集合

2.3 并查集 (union find)

并查集的合并

直接将一个集合的树根连接到另一个集合中即可。



查询元素所在集合

用元素所在集合的树根标记这个集合即可。查询树根每次往上查找。

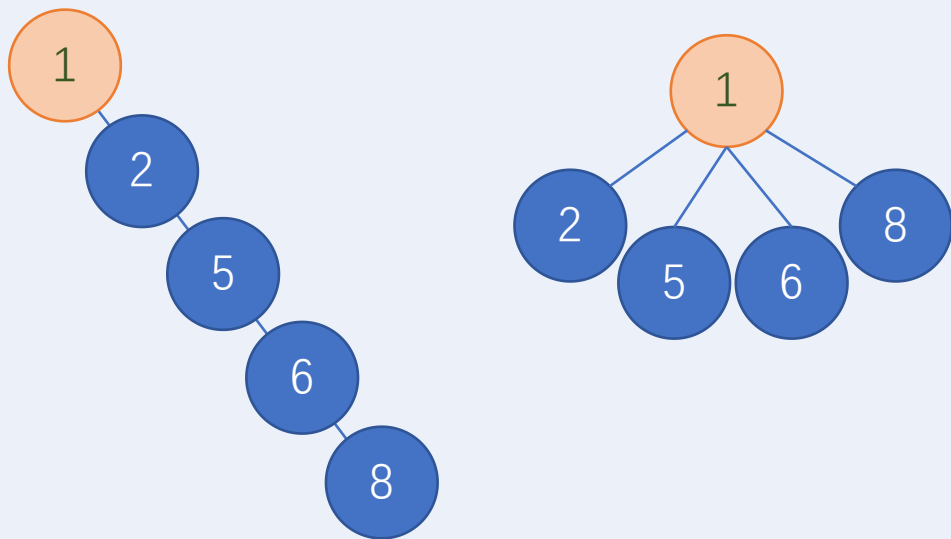
注意到在这两个操作中，我们只需要寻找每个节点所在根节点是否相同，所有不需要存储儿子信息，只对每个节点记录父亲即可。使用数组记录父亲，特别地，设根节点的父亲仍为根节点。

多次查询树根时存在重复查找！

2.3 并查集 (union find)

路径压缩

每次查询根的时候把路上经过的节点的父亲直接设为根节点。



路径压缩并查集的时间复杂度

并查集在同时使用路径压缩和按秩合并（小并大）优化时，单次查询、合并的时间复杂度为 $O(\log n)$ ， n 次操作复杂度为 $O(n\alpha(n))$ ，其中 $\alpha(n)$ 是反阿克曼常数，它增长极缓慢，当 $n \leq 10^{600}$ 时 $\alpha(n) \leq 4$ ，故可忽略此项，认为 n 次操作复杂度为 $O(n)$ 。

2.3 并查集 (union find)

并查集的实现

主流语言中没有实现并查集的STL容器或标准库，需要手动实现。

```
const int MAX = 10004;
struct UnionFind
{
    int N;
    int arr[MAX];

    UnionFind(int n)
    {
        N = n;
        for (int i = 0; i < n; ++i)
        {
            arr[i] = i;
        }
    }
}
```

```
int find(int x)
{
    if (arr[x] != arr[arr[x]])
    {
        arr[x] = find(arr[x]);
    }
    return arr[x];
}

void merge(int x, int y)
{
    arr[find(y)] = find(x);
}

};
```

03



西安交通大学
XI'AN JIAOTONG UNIVERSITY

区间相关数据结构

- 前缀和与差分 (prefix sum and difference array)
- 树状数组 (binary indexed tree)
- ST表/稀疏表 (sparse table)
- 线段树 (segment tree)

3.1 前缀和与差分 (prefix sum and difference array)

问题：区间求和，无修改。

区间长度 $n \leq 2 \times 10^5$

求和次数 $q \leq 2 \times 10^5$

前缀和即数组的前 n 项和，对数组求一次

前缀和可以大大减少重复的求和操作。

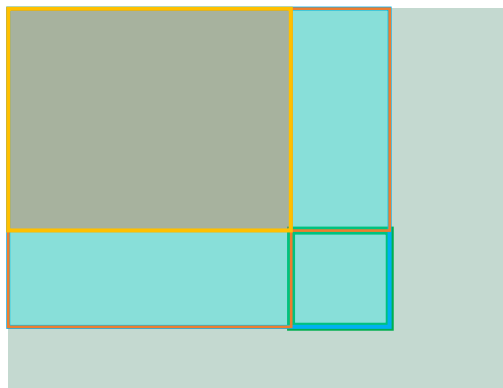
若 $S_0 = 0, S_n = a_1 + a_2 + \dots + a_n$ ，则有

$$a_m + \dots + a_n = S_n - S_{m-1}$$



3.1 前缀和与差分 (prefix sum and difference array)

二维前缀和



$$S_{n_1, n_2} - S_{n_1, m_2-1} - S_{m_1-1, n_2} + S_{m_1-1, m_2-1}$$

3.1 前缀和与差分 (prefix sum and difference array)

差分是前缀和的逆运算，即和前一项的差值。

$$d_1 = a_1, d_i = a_i - a_{i-1}$$

例：数组 a 长度为 n ，初始值都为0， q 次操作，每次操作将 $a[l_i..r_i]$ 都增加 x_i ，求最终 a 的每一位。

$(n \leq 2 \times 10^4, q \leq 2 \times 10^4)$ (leetcode 1109)

在原数组上修改次数太多，考虑其差分数组，每次操作相当于在 d_{l_i} 位置加上 x_i ，在 d_{r_i+1} 位置减去 x_i ，最后求前缀和恢复即可。时间复杂度 $O(n + q)$

3.2 树状数组 (binary indexed tree)

问题：区间求和，带单点修改（增加、减少、赋值）。

区间长度 $n \leq 2 \times 10^5$

求和+修改次数 $q \leq 2 \times 10^5$

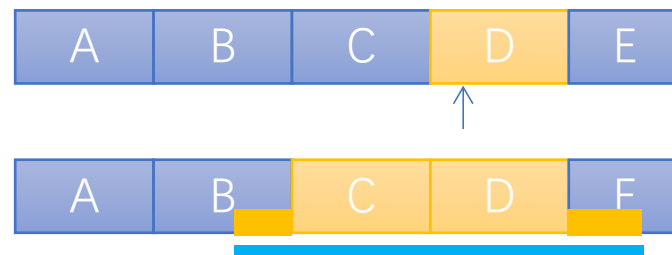
直接算：修改 $O(1)$ 求和 $O(n)$

前缀和：修改 $O(n)$ 求和 $O(1)$

均会超时！需要其他数据结构。

是否有一种两种方案折衷的数据结构？

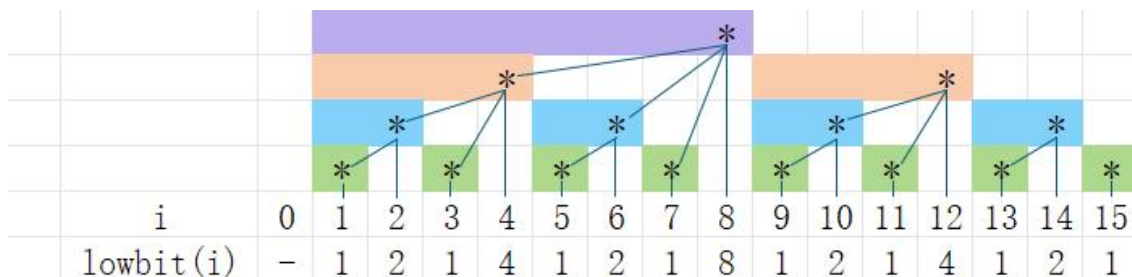
其中一个思路是将数组分成一系列大小为 $O(\sqrt{n})$ 的块，预先计算每一块的和。修改时只需重新计算所在块的和，单次时间复杂度为 $O(\sqrt{n})$ 或 $O(1)$ ，查询时对完整的块以及左右两端的零散数据求和，时间复杂度 $O(\sqrt{n})$ 。这种算法称为分块 (square root decomposition)，总时间复杂度 $O(n + q\sqrt{n})$ 。这里不详细展开。



3.2 树状数组 (binary indexed tree)

树状数组是一个支持单点修改，前缀和查询的数据结构，数组第 i 个位置存储 $[i - \text{lowbit}(i) + 1, i]$ 的元素之和。其中 $\text{lowbit}(i)$ 表示 i 的最低位1对应的值。

- 单点修改 $O(\log n)$
- 前缀和查询 $O(\log n)$



$\text{lowbit}(x)$ 的计算方法:

$$\text{lowbit}(x) = x \& (-x)$$

回顾负数的补码表示，取反加1:

x 的二进制表示: 1010...01000

取反: 0101...10111

取反加1: 0101...11000

3.2 树状数组 (binary indexed tree)

树状数组第 i 位保存了存以它为根的子树

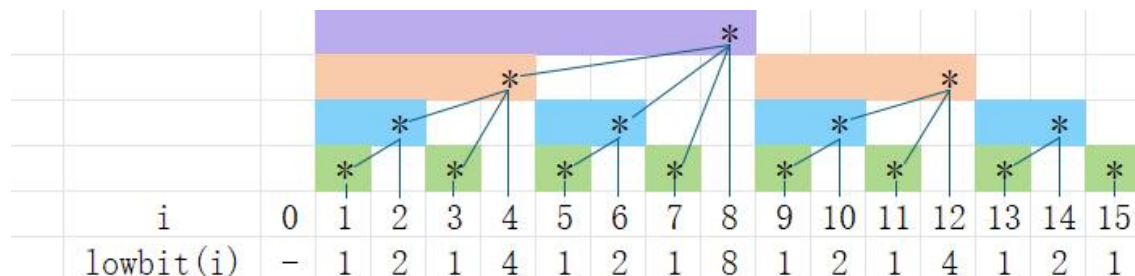
中所有子节点+当前位的和;

第 i 位的子节点个数是 $\log_2 \text{lowbit}(i)$;

除了根节点, 第 i 位的父节点是第 $i +$

$\text{lowbit}(i)$ 位;

树的深度为 $O(\log n)$ 。



单点修改: 需修改这个点及其所有祖先。

```
const int MAX = 10004;
int N;
int fw[MAX];

inline int lowbit(int x)
{
    return x & (-x);
}

void config(int index, int val)
{
    while (index <= N)
    {
        fw[index] += val;
        index += lowbit(index);
    }
}
```

3.2 树状数组 (binary indexed tree)

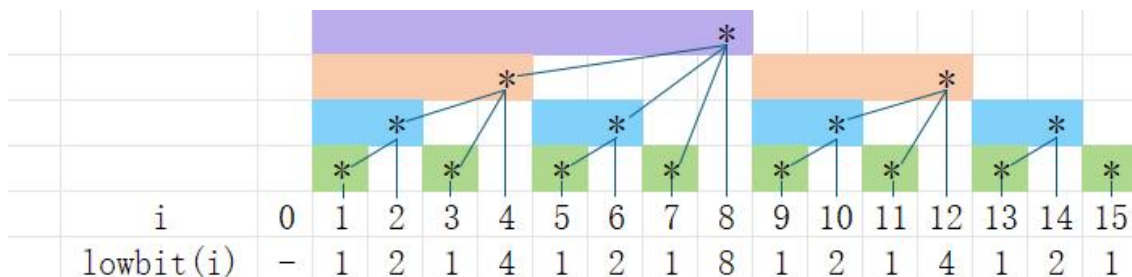
前缀和查询：

$$pf[i] = fw[i] + pf[i - lowbit(i)]$$

每次将 i 减少 $lowbit(i)$ 。

树状数组初始化：

- 一位一位地修改 $O(n \log n)$;
- $fw[i] = pf[i] - pf[i - lowbit(i)]$ $O(n)$



$$0 \leq a_i \leq 10^9$$

```
const int MAX = 10004;
int N;
long long fw[MAX];

inline int lowbit(int x)
{
    return x & (-x);
}

void config(int index, int val)
{
    while (index <= N)
    {
        fw[index] += val;
        index += lowbit(index);
    }
}

long long search(int index)
{
    long long res = 0;
    while (index > 0)
    {
        res += fw[index];
        index -= lowbit(index);
    }
    return res;
}
```

3.2 树状数组 (binary indexed tree)

例：逆序对（洛谷P1908弱化版）

给定一个 n 的排列，求满足 $i > j$ 且 $a_i < a_j$ 的数对 (i, j) 的个数， $n \leq 5 \times 10^5$ 。

处理完小于 j 的所有数后，能否快速计算对于特定 j 时 (i, j) 的个数？

使用树状数组维护处理到第 j 位时值域计数的前缀和，处理每一位时计数并调整即可。

如果序列满足 $0 \leq a_i \leq 10^9$ 且有重复（洛谷P1908原版）怎么办？

3.3 ST表 (稀疏表, sparse table)

问题：区间求最大/最小值，无修改。

区间长度 $n \leq 2 \times 10^5$

求值次数 $q \leq 2 \times 10^5$

最大值和最小值不具有消去律，无法用前缀和相减计算。

分块可以 $O(n + q\sqrt{n})$ 解决这个问题。

最大最小值满足幂等律， $\max(x, x) = x$

——这允许我们重复覆盖某些元素



两条长度均为 n 的线段可覆盖连续长度的范围是？

我们只需要储存起始为 i 节点，长度为 $2^k (k \leq \log_2 n)$ 的区间的最大/最小值即可。

——倍增

- 初始化 $O(n \log n)$ 时间、 $O(n \log n)$ 空间
- 最大最小值查询 $O(1)$ 时间

3.3 ST表 (稀疏表, sparse table)

ST表的实现

```
const int MAX = 10004;
int ST[MAX][14];

int main()
{
    int n;
    cin >> n;
    for (int i = 0; i < n; ++i)
    {
        cin >> ST[i][0];
    }
    for (int j = 1; j < 14; ++j)
    {
        for (int i = 0; i + (1 << j) <= n; ++i)
        {
            ST[i][j] = max(ST[i][j - 1], ST[i + (1 << (j - 1))][j - 1]);
        }
    }
}

int search(int a, int b) // 左闭右开
{
    int d = __lg(b - a);
    return max(ST[a][d], ST[b - (1 << d)][d]);
}
```

3.4 线段树 (segment tree)

刚才我们针对不同的问题，使用了不同的数据结构，有没有一种数据结构能平均

$O(\log n)$ 地处理上述所有问题？

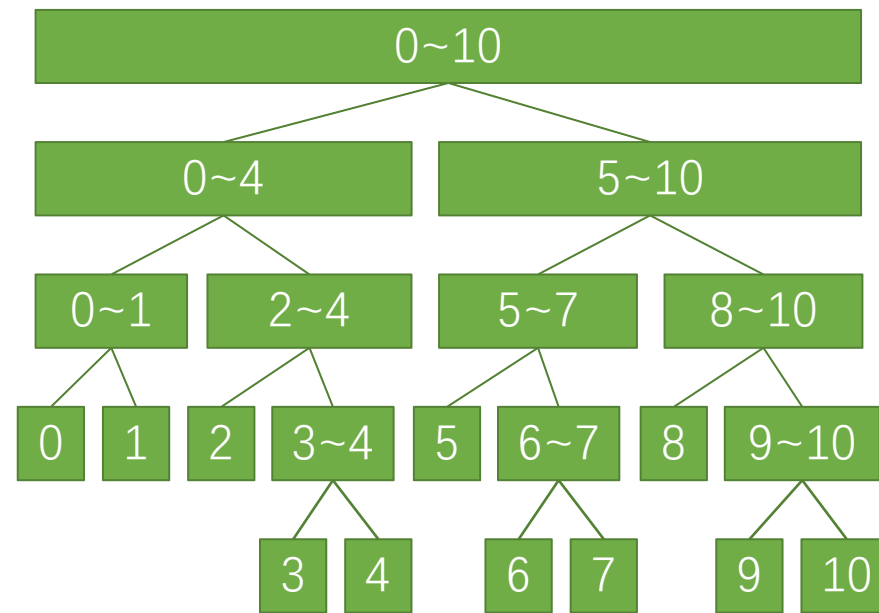
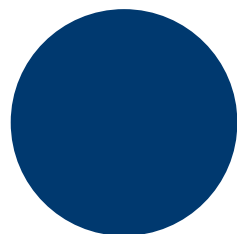
如何处理区间求最大/最小值，带修改问题？



3.4 线段树 (segment tree)

线段树是在“半群”和“半群作用”这两个代数结构上，通过二叉分解实现区间查询与修改的数据结构。

将每个长度不为 1 的区间划分成左右两个区间递归求解，根节点维护的是整个区间，每个节点维护的是父亲的区间二等分后的一个子区间。把整个线段划分为一个二叉树结构，通过合并左右两区间信息来求得该区间的信息。



每个节点要么没有子节点，要么有2个子节点，不可能有1个子节点。

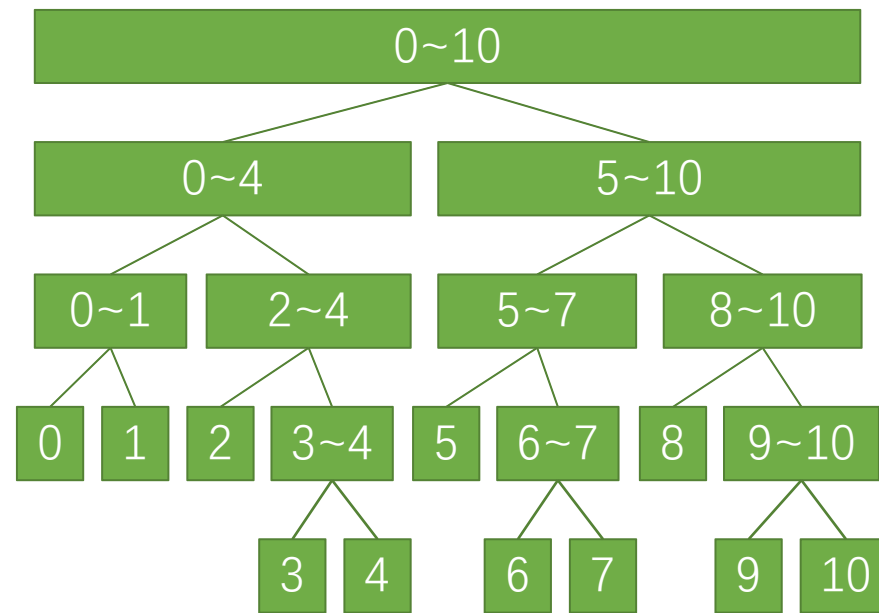
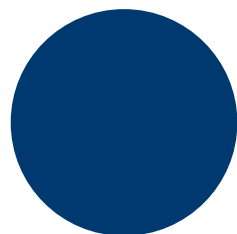
3.4 线段树 (segment tree)

以区间求和为例，我们使用线段树来维护当前节点对应段的和。

对于任一区间，可以用不超过 $2\log n$ 个不相交的区间（节点）将其恰好覆盖。

例如区间 $[2, 7)$

实现查找操作：从根节点出发进行递归
 $O(\log n)$



3.4 线段树 (segment tree)

递归的4种情况

(1)要查询的区间为空，直接返回0。

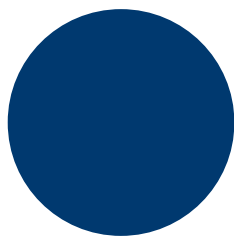
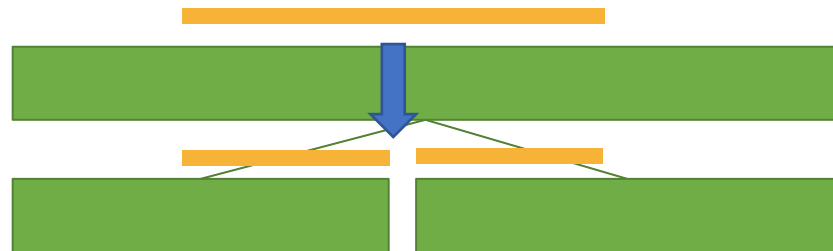
(2)要查询的区间恰好为这个区间本身，返回这个区间的结果。



(3)要查询的区间完全包含在这个区间的左子节点（或右子节点）里，递归查找这个节点。



(4)要查询的区间跨越了左右子节点，递归查找左右节点后再合并。



3.4 线段树 (segment tree)

对于单点修改操作，方法类似，修改子节点
完回溯时需更新父节点。

线段树的结构体实现

```
struct SegTree
{
    int l, m, r;
    int val = 0;
    SegTree* left = nullptr, * right = nullptr;

    void up()
    {
        if (r - l > 1)
        {
            val = left->val + right->val;
        }
    }
};
```

```
SegTree* create_tree(int l, int r) // 初始化
{
    SegTree* t = new SegTree;
    t->l = l, t->r = r, t->m = (l + r) >> 1;
    if (r - l > 1)
    {
        t->left = create_tree(l, t->m);
        t->right = create_tree(t->m, r);
    }
}
```

```
void config(SegTree* t, int index, int val) // 单点修改
{
    if (t->r - t->l == 1)
    {
        t->val += val;
    }
    else if (index >= t->m)
    {
        config(t->right, index, val);
        t->up();
    }
    else
    {
        config(t->left, index, val);
        t->up();
    }
}
```

3.4 线段树 (segment tree)

线段树的数组实现

```
const int MAX = 10004;
struct SegTree
{
    int l, m, r;
    int val = 0;
    // 节点 i 的左子节点为 2*i+1, 右子节点为 2*i+2

    static void up(SegTree t[], int index)
    {
        if (t[index].r - t[index].l == 1)
        {
            t[index].val = t[2 * index + 1].val + t[2 * index + 2].val;
        }
    }
} entry[MAX * 4]; // 4倍空间!

void create_tree(SegTree t[], int l, int r, int index = 0) // 初始化
{
    t[index].l = l, t[index].r = r, t[index].m = (l + r) >> 1;
    if (r - l > 1)
    {
        create_tree(t, l, t[index].m, 2 * index + 1);
        create_tree(t, t[index].m, r, 2 * index + 2);
    }
}
```

大小为 n 的线段树有 $2n-1$ 个节点，但这些节点不集中在前面 $2n-1$ 个位置处。

```
void config(SegTree t[], int idx, int val, int index = 0) // 单点修改
{
    if (t[index].r - t[index].l == 1)
    {
        t[index].val += val;
    }
    else if (idx >= t[index].m)
    {
        config(t, idx, val, 2 * index + 2);
        SegTree::up(t, index);
    }
    else
    {
        config(t, idx, val, 2 * index + 1);
        SegTree::up(t, index);
    }
}
```

3.4 线段树 (segment tree)

```
int search(SegTree t[], int l, int r, int index = 0) // 区间查询
{
    if (l == t[index].l && r == t[index].r)
    {
        return t[index].val;
    }
    else if (l >= t[index].m)
    {
        return search(t, l, r, 2 * index + 2);
    }
    else if (r <= t[index].m)
    {
        return search(t, l, r, 2 * index + 1);
    }
    else
    {
        return search(t, l, t[index].m, 2 * index + 1) + search(t, t[index].m, r, 2 * index + 2);
    }
}
```

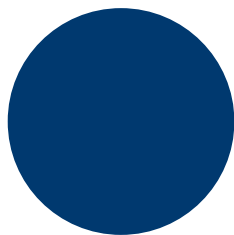
3.4 线段树 (segment tree)

问题：区间求和，带区间修改（增加、减少、赋值）。

区间长度 $n \leq 2 \times 10^5$

求和+修改次数 $q \leq 2 \times 10^5$

修改区间所有节点的时间复杂度为 $O(n)$



有很多节点的信息在修改后暂时不需要用到，考虑先打上一个标记，要用到时再更新。

这个标记称为懒标记 (**lazy-tag**)，更新的操作称为下传 (**pushdown**)。

```
static void dwn(SegTree t[], int index)
{
    if (t[index].r - t[index].l > 1)
    {
        t[2 * index + 1].val += t[index].lazy * (t[index].m - t[index].l);
        t[2 * index + 1].lazy += t[index].lazy;
        t[2 * index + 2].val += t[index].lazy * (t[index].r - t[index].m);
        t[2 * index + 2].lazy += t[index].lazy;
    }
    t[index].lazy = 0;
}
```

3.4 线段树 (segment tree)

struct SegTree

```
{
    int l, m, r;
    int val = 0;
    int lazy = 0;
    // 节点 i 的左子节点为2i+1, 右子节点为2i+2

    static void up(SegTree t[], int index)
    {
        if (t[index].r - t[index].l > 1)
        {
            t[index].val = t[2 * index + 1].val + t[2 * index + 2].val;
        }
    }

    static void dwn(SegTree t[], int index)
    {
        if (t[index].r - t[index].l > 1)
        {
            t[2 * index + 1].val += t[index].lazy * (t[index].m - t[index].l);
            t[2 * index + 1].lazy += t[index].lazy;
            t[2 * index + 2].val += t[index].lazy * (t[index].r - t[index].m);
            t[2 * index + 2].lazy += t[index].lazy;
        }
        t[index].lazy = 0;
    }
} entry[MAX * 4]; // 4倍空间!
```

```
void create_tree(SegTree t[], int l, int r, int index = 0) // 初始化
{
    t[index].l = l, t[index].r = r, t[index].m = (l + r) >> 1;
    if (r - l > 1)
    {
        create_tree(t, l, t[index].m, 2 * index + 1);
        create_tree(t, t[index].m, r, 2 * index + 2);
    }
}

void config(SegTree t[], int l, int r, int val, int index = 0) // 单点修改
{
    if (l == t[index].l && r == t[index].r)
    {
        t[index].val += val * (r - l);
        t[index].lazy += val;
    }
    else if (l >= t[index].m)
    {
        SegTree::dwn(t, index);
        config(t, l, r, val, 2 * index + 2);
        SegTree::up(t, index);
    }
    else if (r <= t[index].m)
    {
        SegTree::dwn(t, index);
        config(t, l, r, val, 2 * index + 1);
        SegTree::up(t, index);
    }
    else
    {
        SegTree::dwn(t, index);
        config(t, l, t[index].m, val, 2 * index + 1);
        config(t, t[index].m, r, val, 2 * index + 2);
        SegTree::up(t, index);
    }
}
```


3.4 线段树 (segment tree)

```
int search(SegTree t[], int l, int r, int index = 0) // 区间查询
{
    if (l == t[index].l && r == t[index].r)
    {
        return t[index].val;
    }
    else if (l >= t[index].m)
    {
        SegTree::dwn(t, index);
        return search(t, l, r, 2 * index + 2);
    }
    else if (r <= t[index].m)
    {
        SegTree::dwn(t, index);
        return search(t, l, r, 2 * index + 1);
    }
    else
    {
        SegTree::dwn(t, index);
        return search(t, l, t[index].m, 2 * index + 1) + search(t, t[index].m, r, 2 * index + 2);
    }
}
```

3.4 线段树 (segment tree)

处理区间赋值要注意什么？

注意到刚才处理区间加时，下传时+0的操作等价于没有传。

赋值操作没有单位元，区间赋值中“当前不需下传”的懒标记需使用题目规定值域外的一个数。下传时需要特判。

处理多个区间操作时要注意什么？

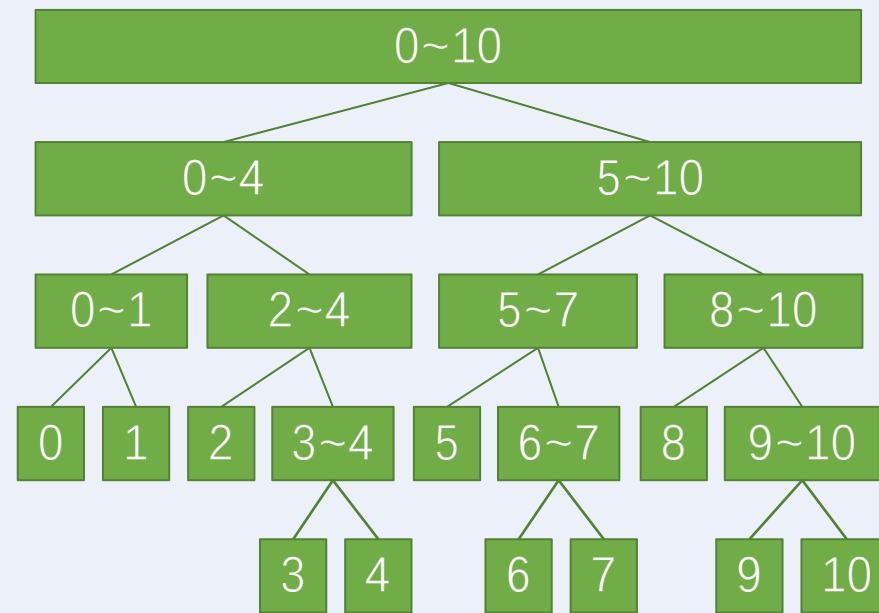
如果有区间加、乘、赋值多个操作时，需要多个懒标记。在执行一个操作修改节点的值时，注意其他操作的懒标记也要修改。

例如，某个区间 $+x$ 并打上了懒标记，现在将区间的一部分 $\times y$ ，则加法懒标记也要 $\times y$ 。

3.4 线段树 (segment tree)

线段树的缺点

- 需开4倍空间，空间开销较大；
- 实现和维护复杂，调试困难；
- 查询和修改的速度稍慢。



3.5 总结

数据结构	A1	A2	A3	B1	B2	B3	C1	C2	C3
前缀和/差分	$O(1)$								
树状数组	$O(\log n)$	$O(\log n)$	部分 $O(\log n)$						
ST表				$O(1)$					
线段树	$O(\log n)$	$O(\log n)$	$O(\log n)$	$O(\log n)$	$O(\log n)$	$O(\log n)$	$O(\log n)$	$O(\log n)$	$O(\log n)$

A: 满足结合律、消去律的运算，如加法、乘法（不含0）、按位异或等；

B: 满足结合律、幂等律的运算，如最大/最小值、按位与/或、gcd、lcm等；

C: 仅满足结合律的运算

1: 仅区间查询

2: 单点修改，区间查询

3: 区间修改，区间查询

课后练习

- 洛谷P3373 (【模板题】线段树2, 练习区间乘加操作线段树)
- 洛谷P4053 (建筑抢修)
- 洛谷P4086 (My Cow Ate My Homework S)
- 洛谷P4784 (Hacker)
- Q0J 1981 (Safe Distance)

其他常见数据结构

- 单调队列
- 单调栈
- 动态开点线段树
- 可持久化线段树
- 线段树/树状数组上二分
- 字典树 (Trie树)
- 可撤销并查集
- 分块算法
- 莫队算法
- 树的重链剖分
- 点分治

- 树上启发式合并
- 动态树LCT



谢谢大家!

// Austin4.5

// 2026.7.13

